

Building a RAG Chatbot with LangChain

A step-by-step guide to a document question-answering project using LangChain, OpenAI, and FAISS.

This guide walks through a complete Retrieval-Augmented Generation (RAG) project: a chatbot that answers questions about your own documents (a company handbook, in this example) instead of relying only on the model's general knowledge.

What you'll build

- A pipeline that loads and chunks text documents
- A FAISS vector store of embedded document chunks
- A retrieval chain that finds relevant chunks and answers questions with an LLM
- A simple command-line chat loop to ask questions

Project structure

```
langchain-rag-qa/  
■ requirements.txt  
■ .env.example  
■ data/  
■ ■ handbook.txt          (your source documents go here)  
■ ingest.py                (builds the vector store)  
■ ask.py                   (runs the Q&A; chat loop)
```

Step-by-Step Procedure

Step 1 — Install prerequisites

You need Python 3.9+ and an OpenAI API key. Create a project folder and a virtual environment:

```
mkdir langchain-rag-qa && cd langchain-rag-qa
python -m venv venv
source venv/bin/activate # Windows: venv\Scripts\activate
```

Step 2 — Install dependencies

Create requirements.txt with the packages below, then install them.

```
langchain==0.3.7
langchain-openai==0.2.9
langchain-community==0.3.7
faiss-cpu==1.9.0
python-dotenv==1.0.1
pypdf==5.1.0
tiktoken==0.8.0

pip install -r requirements.txt
```

Step 3 — Set your API key

Create a .env file in the project root (never commit this file) with your OpenAI key:

```
OPENAI_API_KEY=sk-...your-key-here...
```

Step 4 — Add your source documents

Create a data/ folder and drop in the text you want the chatbot to answer questions about (handbook, FAQ, policy doc, etc.) as .txt files. This example uses a sample company handbook.

Step 5 — Write the ingestion script (ingest.py)

This script loads documents, splits them into overlapping chunks, embeds each chunk with OpenAI embeddings, and saves the result as a local FAISS index.

```
loader = DirectoryLoader("data", glob="**/*.txt", loader_cls=TextLoader)
documents = loader.load()

splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=80)
chunks = splitter.split_documents(documents)

embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
vectorstore = FAISS.from_documents(chunks, embeddings)
vectorstore.save_local("vectorstore")
```

Chunk size and overlap control how much context each embedded piece carries — smaller chunks retrieve more precisely, larger chunks preserve more context.

Step 6 — Run the ingestion script

This builds the vectorstore/ folder. Run it once, and again any time data/ changes.

```
python ingest.py
```

Step 7 — Write the retrieval chain (ask.py)

Load the saved FAISS index, wrap it as a retriever, and connect it to a prompt and chat model using LangChain's pipe syntax (LCEL).

```
retriever = vectorstore.as_retriever(search_kwargs={"k": 3})
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
prompt = ChatPromptTemplate.from_template(PROMPT_TEMPLATE)

chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | llm
    | StrOutputParser()
)
```

k=3 retrieves the 3 most relevant chunks for each question and inserts them into the prompt as context.

Step 8 — Run the chatbot

Start the chat loop and ask questions about your documents:

```
python ask.py
```

You: How many PTO days do employees get?

Assistant: Full-time employees accrue 15 days of paid time off per year...

Step 9 — Extend it

A few natural next steps once the base pipeline works:

- Swap DirectoryLoader for PyPDFLoader to ingest PDFs instead of .txt files
- Add chat memory so follow-up questions retain context
- Wrap ask.py in a Flask/FastAPI endpoint or a Streamlit UI
- Swap FAISS for a hosted vector DB (Pinecone, Chroma, Weaviate) for production

How the Pipeline Works

Retrieval-Augmented Generation combines a search step with a generation step, so the model answers using your documents instead of guessing from memory.

Stage	What happens
Load	Documents are read from disk into LangChain Document objects.
Split	Long documents are broken into overlapping chunks so each piece fits in context.
Embed	Each chunk is converted into a vector that captures its meaning.
Store	Vectors are saved in a FAISS index for fast similarity search.
Retrieve	A question is embedded and compared to stored vectors to find the closest chunks.
Generate	The retrieved chunks and the question are passed to the LLM, which answers using only that context.

Troubleshooting

- **ModuleNotFoundError** — re-run `pip install -r requirements.txt` inside the activated virtual environment.
- **AuthenticationError** — check that `.env` has a valid `OPENAI_API_KEY` and `load_dotenv()` runs before the client is created.
- **No vector store found** — run `python ingest.py` before `python ask.py`.
- **Answers ignore your documents** — lower `chunk_size` or increase `k` so more relevant context reaches the model.